

RBE549: Project 2 - Buildings built in minutes - SfM and NeRF

Shreyas Devdatta Khobragade
MS in Robotics Engineering
Worcester Polytechnic Institute
skhobragade@wpi.edu

Neel Girish Bahadarpurkar
MS in Robotics Engineering
Worcester Polytechnic Institute
nbahadarpurkar@wpi.edu

I. PHASE 1 : CLASSICAL SfM

A. Introduction

Structure from Motion (SfM) is a computer vision technique that allows for the reconstruction of three-dimensional scenes and camera stances from a collection of two-dimensional photographs. This technology enables the development of comprehensive 3D models from numerous perspectives. SfM has transformed subjects ranging from archeology to urban planning, with prominent applications like as reconstructing entire cities from photo collections, as demonstrated by Agarwal et al. in their seminal work "Building Rome in a Day".

- Feature Matching and Outlier rejection using RANSAC.
- Estimating Fundamental Matrix.
- Estimating Essential Matrix from Fundamental Matrix.
- Estimate Camera Pose from Essential Matrix.
- Check for Cheirality Condition using Triangulation
- Optimize using Non-Linear Triangulation.
- Perspective-n-Point.
- Optimization using Non-Linear Perspective-n-Point.
- Bundle Adjustment.

These components work together to convert a collection of separated images into a cohesive 3D structure, successfully simulating a camera's motion through a static scene and extracting depth information from the apparent movement of objects across numerous frames.

B. Data Preparation

The dataset for this Structure from Motion project was provided to us, which included preprocessed images of Unity Hall at WPI, as well as critical calibration and feature matching information. Images acquired at 4000×3000 pixels were reduced to 800×600 pixels and adjusted for distortion. This preprocessed data consists of five images, a camera calibration file, and matching files with SIFT feature correspondences between image pairings.

We developed an effective code for reading and organizing the provided data. Our `read_images` function loads the PNG images in their proper order. We utilized the `read_intrinsic_matrix` function to obtain the camera's intrinsic parameters from the calibration file. To process the matching information, we created the `read_matching_files` function, which parses the text files and builds a dictionary. For this

dictionary, we used picture pair indices as keys and arrays of matching feature coordinates as values. By arranging our data loading and categorization in this manner, we guaranteed that all relevant elements were easily accessible for the next phases in our SfM pipeline, laying the groundwork for future processing.

C. Estimating Fundamental Matrix

The fundamental matrix (F) is an important mathematical representation in epipolar geometry that connects related spots in two images taken from different perspectives. It records the inherent projective geometry between views based solely on the cameras' internal characteristics and relative posture, regardless of scene organization. The matrix meets the epipolar constraint:

$$x_2^T F x_1 = 0$$

, where x_1 and x_2 correspond to points in the two images. To estimate F, we employ the eight-point algorithm, which generates a homogeneous linear system from at least eight correspondences.

$$[x'_i \ y'_i \ 1] \begin{bmatrix} f_{11} & f_{21} & f_{31} \\ f_{12} & f_{22} & f_{32} \\ f_{13} & f_{23} & f_{33} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = 0$$

Expanding this equation gives:

$$\begin{aligned} & x_i x'_i f_{11} + x_i y'_i f_{21} + x_i f_{31} \\ & + y_i x'_i f_{12} + y_i y'_i f_{22} + y_i f_{32} \\ & + x'_i f_{13} + y'_i f_{23} + f_{33} = 0 \end{aligned}$$

Simplifying for m correspondences, we construct a matrix A :

$$\begin{bmatrix} x_1 x'_1 & \cdots & y'_1 y_m & 1 \\ \vdots & \ddots & \vdots & \vdots \\ x_m x'_m & \cdots & y'_m y_m & 1 \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{21} \\ \vdots \\ f_{33} \end{bmatrix} = 0$$

Solving this system with Singular Value Decomposition (SVD) ensures that F satisfies the rank-2 constraint, which is enforced by setting the smallest singular value to zero.

Given noisy data, we used RANSAC (Random Sample Consensus) to robustly estimate F by iteratively picking random

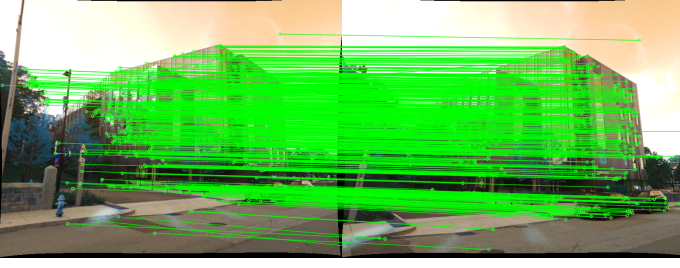


Fig. 1. RANSAC Matches

subsets of correspondences, estimating a candidate F , and identifying inliers using the epipolar error.

$$|x_2^T F x_1| < \epsilon$$

The optimal F is the one that maximizes inliers. Our implementation normalizes points for numerical stability, creates a matrix A for the eight-point technique, and refines F with SVD. This approach ensures an accurate and robust estimation of the basic matrix, which is required for following tasks such as camera position estimation and 3D reconstruction in Structure from Motion.

The RANSAC matching are displayed in Figure 1.

D. Estimating Essential Matrix

The essential matrix (E) is a 3×3 matrix that connects corresponding spots between two images, provided the cameras use the pinhole paradigm. Unlike the fundamental matrix (F), which uses pixel coordinates, E uses normalized image coordinates, with the origin at the optical center. The essential matrix is calculated from F and the camera intrinsic matrix (K) using the following relationship:

$$E = K^T F K$$

Here, K represents the calibration matrix including the camera's inherent properties. Because of noise in K or F , the singular values of E may differ from their ideal values of $(1, 1, 0)$. To address this, we reconstruct E by enforcing these unique values using unique Value Decomposition (SVD).

$$E = U \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} V^T$$

This assures that E meets its mathematical requirements. Importantly, F has seven degrees of freedom, whereas E only has five due to camera characteristics. The fundamental matrix is critical for predicting relative camera positions between two views, laying the groundwork for tasks such as motion estimation and 3D reconstruction.

The Epipolar Lines Plot is shown in Figure 2.

E. Estimating Camera Pose

The estimation of camera pose from the Essential Matrix (E) is an important step in Structure from Motion because it provides the camera's 6 degrees of freedom (DOF) orientation

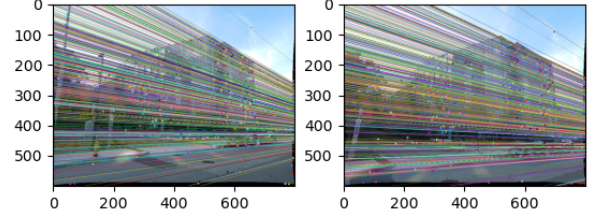


Fig. 2. Epipolar Lines

and location relative to the world coordinate system. This algorithm extracts four alternative camera pose configurations from E , each consisting of a rotation matrix $R \in SO(3)$ and a camera center $C \in R^3$. The camera pose can be represented as follows:

$$P = KR[I_{3 \times 3} | -C]$$

where K is the intrinsic camera matrix. Given the singular value decomposition of $E = UDV^T$, and defining W as:

$$W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The four possible configurations are computed as:

$$\begin{array}{ll} C_1 = U(:, 3) & \text{and} \quad R_1 = U W V^T \\ C_2 = -U(:, 3) & \text{and} \quad R_2 = U W V^T \\ C_3 = U(:, 3) & \text{and} \quad R_3 = U W^T V^T \\ C_4 = -U(:, 3) & \text{and} \quad R_4 = U W^T V^T \end{array}$$

It is critical to verify that $\det(R) = 1$ in each arrangement. If $\det(R) = -1$, the posture is adjusted by negating C and R . This approach generates four possible camera postures, from which the proper one must be selected using processes such as triangulation and cheirality checks.

F. Linear Triangulation and Cheirality Condition

Triangulation and the Cheirality Condition are critical phases in Structure from Motion (SfM) when resolving camera pose ambiguity and recreating 3D points. After computing four potential camera positions from the fundamental matrix, we use linear triangulation to reconstruct 3D points given two camera poses, (C_1, R_1) and (C_2, R_2) , and their corresponding image points, $x_1 \leftrightarrow x_2$.

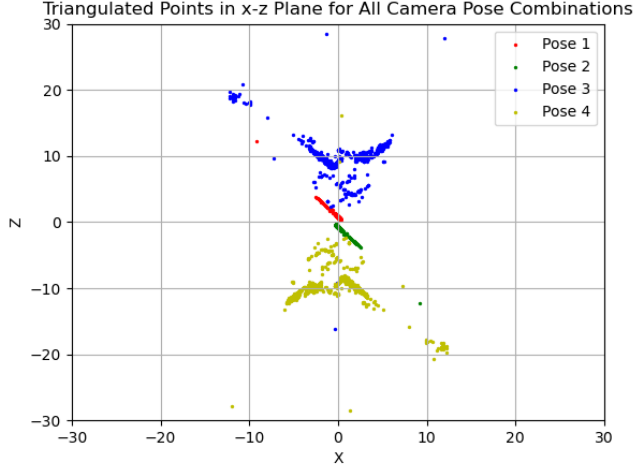


Fig. 3. Linear Triangulation

The linear triangulation method resolves the following system for each point:

$$\begin{bmatrix} x_1 P_1^{3T} - P_1^{1T} \\ y_1 P_1^{3T} - P_1^{2T} \\ x_2 P_2^{3T} - P_2^{1T} \\ y_2 P_2^{3T} - P_2^{2T} \end{bmatrix} X = 0$$

P_1 and P_2 are the two cameras' projection matrices, and P_i^{jT} represents the j -th row of P_i .

To disambiguate the camera attitude, we use the Cheirality criterion, which requires that reconstructed points be in front of both cameras. Mathematically, a 3D point X that is in front of a camera with center C and rotation R must satisfy:

$$r_3(X - C) > 0$$

We assess this condition for all four feasible camera postures and associated triangulated sites. The best camera setup (C, R, X) maximizes the number of spots that satisfy the Cheirality requirement for both cameras. This technique effectively resolves pose ambiguity and generates a unique camera posture as well as a set of valid 3D points, laying the groundwork for the next phases in the SfM pipeline.

The output for Linear Triangulation is shown in Figure 3.

G. Non-Linear Triangulation

We used a non-linear triangulation method to refine the 3D point locations that were first predicted using linear triangulation. Our technique minimizes the reprojection error, which yields a more geometrically significant optimization than the algebraic error minimized in linear triangulation.

Our implementation aims to minimize the following cost function:

$$\min_X \sum_{j=1,2} \left(\left(u_j - \frac{P_j^T 1 \tilde{X}}{P_j^T 3 \tilde{X}} \right)^2 + \left(v_j - \frac{P_j^T 2 \tilde{X}}{P_j^T 3 \tilde{X}} \right)^2 \right)$$

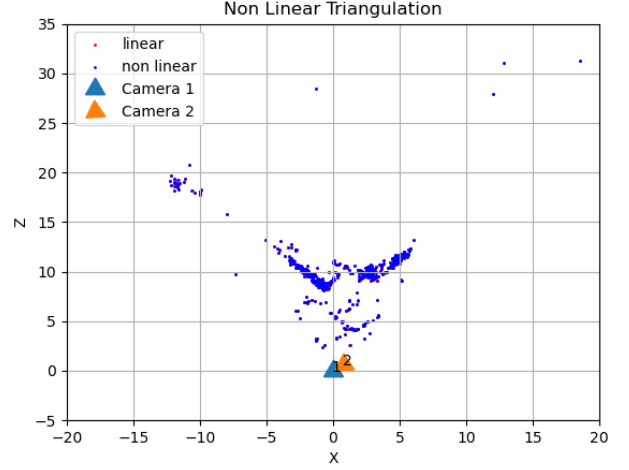


Fig. 4. Non Linear Triangulation

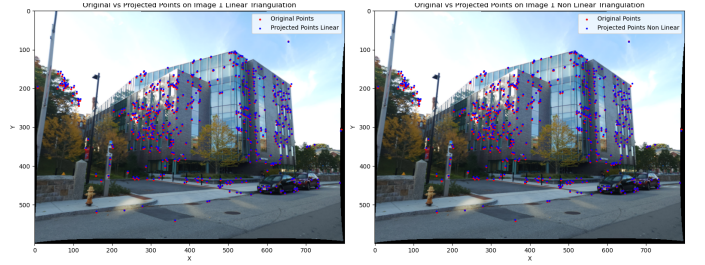


Fig. 5. Projected points after Linear vs Non Linear Triangulation

where j indexes each camera; $\tilde{X}$ is the homogeneous representation of the 3D point X ; and $P_j^T i$ represents the i -th row of the j -th camera's projection matrix.

We used the SciPy library's `least_squares` function to tackle this nonlinear optimization problem. Our method uses the linearly triangulated locations as starting estimations and iteratively refines them to reduce reprojection errors. This method often yields more accurate 3D point estimates, as indicated by lower reprojection errors when the points are projected back onto the image planes using the estimated camera positions. Our implementation shows that non-linear optimization improves the accuracy of 3D reconstruction in Structure from Motion pipelines.

The output for Non Linear Triangulation is displayed in Figure 12 and the comparison of projected points after Linear and Non Linear Triangulation is shown in Figure 5

H. Perspective-n-Points

1) *Linear Camera Pose Estimation:* We used the Perspective-n-Point (PnP) approach to estimate the 6 degrees-of-freedom (DOF) camera position, which consists of rotation (R) and translation (t), from a set of n 3D points in the environment and their corresponding 2D projections in an image. Given the intrinsic camera matrix K , PnP uses the camera projection equation to determine a relationship between 3D points X and their 2D projections x .

$$x = K[R|t]X$$

where x is the 2D projection, K is the intrinsic matrix, and $[R|t]$ denotes the extrinsic parameters. To solve for R and t , we normalize the 2D points using $K^{-1}x$ and create a linear least squares system. The solution is obtained using Singular Value Decomposition (SVD). Linear methods do not impose the orthogonality restriction on R , thus it must be rectified as follows:

$$R = UV^T$$

Let $R = UDV^T$. If $\det(R) = -1$, we can negate both R and t . This ensures that $R \in SO(3)$.

2) *PnP RANSAC*: We combined PnP with RANSAC to make it more resilient to outliers. The approach iteratively picks subsets of correspondences, estimates the pose with Linear PnP, and calculates the reprojection error using:

$$e = \sqrt{\left(u - \frac{P_1^T \tilde{X}}{P_3^T \tilde{X}}\right)^2 + \left(v - \frac{P_2^T \tilde{X}}{P_3^T \tilde{X}}\right)^2}$$

where u', v' represent reprojected 2D points. Inliers are defined as correspondences with mistakes that are less than a certain threshold. The best position is determined by the largest number of inliers. Finally, nonlinear optimization can be used to optimize the pose and reduce reprojection error. This approach enables robust camera pose estimation by iteratively refining the pose and removing outliers.

I. Non-Linear PnP

We used a Nonlinear Perspective-n-Point (PnP) technique to improve camera pose estimation. Given $N \geq 6$ 3D-2D correspondences, $X \leftrightarrow x$, and an initial camera pose (C, R) derived using linear PnP, this technique minimizes the reprojection error. This provides a more geometrically relevant optimization than the algebraic error minimized in linear PnP.

This implementation aims to minimize the following cost function:

$$\min_{C, R} \sum_{i=1}^J \left(\left(u_j - \frac{P_j^T 1 \tilde{X}_j}{P_j^T 3 \tilde{X}_j} \right)^2 + \left(v_j - \frac{P_j^T 2 \tilde{X}_j}{P_j^T 3 \tilde{X}_j} \right)^2 \right) \quad (1)$$

where $\tilde{X}_j$ is the homogeneous representation of the 3D point X_j , and $P_j^T i$ represents the i -th row of the camera projection matrix P . We compute P as:

$$P = KR[I_{3 \times 3} | -C] \quad (2)$$

To enforce the orthogonality constraint on the rotation matrix, we used the quaternion representation $R = R(q)$, where q is a four-dimensional quaternion. This leads to the refined optimization problem.

$$\min_{C, q} \sum_{i=1}^J \left(\left(u_j - \frac{P_j^T 1 \tilde{X}_j}{P_j^T 3 \tilde{X}_j} \right)^2 + \left(v_j - \frac{P_j^T 2 \tilde{X}_j}{P_j^T 3 \tilde{X}_j} \right)^2 \right) \quad (3)$$

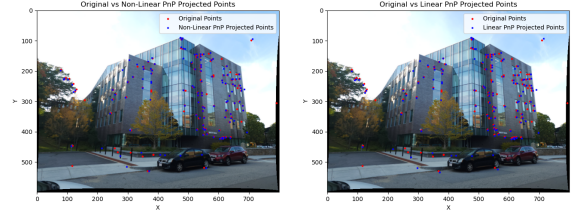


Fig. 6. Projected Points after after Linear vs Non Linear PnP

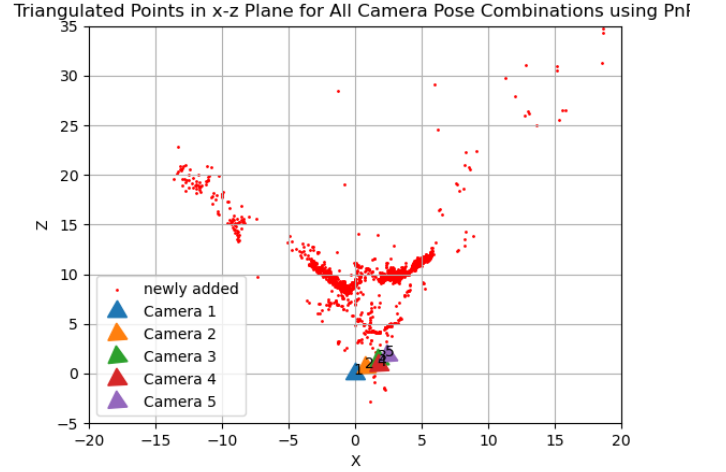


Fig. 7. Camera Poses and Projected Points After Non Linear PnP

We used SciPy's `least_squares` function to tackle this non-linear optimization problem. We use the linearly estimated camera pose (C_0, R_0) as a starting approximation and iteratively refine it to minimize the reprojection error.

The implementation demonstrates how non-linear optimization can improve the quality of camera pose estimation in Structure from Motion pipelines, especially when dealing with noisy correspondences or faulty beginning estimates.

The outputs after Linear vs Non Linear PnP are shown in Figure 6 and the camera poses and projected points after PnP are shown in Figure 7

J. Visibility Matrix

We developed a method for creating the Visibility Matrix, which determines the link between cameras and 3D points in a Structure from Motion (SfM) pipeline. This matrix is critical for determining which points are visible from which cameras, allowing for accurate bundle adjustment and 3D reconstruction.

Our method, encoded in the `BuildVisibilityMatrix` function, builds a $I \times J$ binary matrix V , where I is the number of 3D world points and J is camera poses. The entry V_{ij} is set to 1 when the j -th point is visible from the i -th camera, and 0 otherwise.

Our approach additionally creates a pixel matrix to hold the 2D coordinates of each visible world point in each camera's

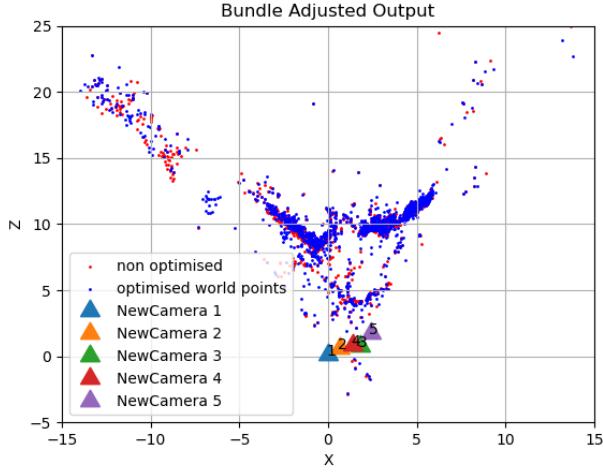


Fig. 8. Final Bundle Adjustment Output

image plane. This additional information is useful for the next phase in the SfM pipeline called bundle adjustment and simplifies the implementation of bundle adjustment while making it efficient by avoiding unnecessary data processing for getting the corresponding pixel coordinates for each image.

The visibility matrix provides a compact representation of the scene’s visibility relationships, allowing for efficient processing in later phases of 3D reconstruction and optimization.

K. Bundle Adjustment

We used a Bundle Adjustment method to improve camera poses and 3D point placements in our Structure from Motion (SfM) workflow. Bundle Adjustment is an important optimization step that reduces reprojection error across all cameras and 3D points. Our approach handles the following optimization problems:

$$\min_{C_i, q_i} \sum_{i=1}^I \sum_{j=1}^J V_{ij} \left(\left(u_j - \frac{P_j^T 1 \tilde{X}}{P_j^T 3 \tilde{X}} \right)^2 + \left(v_j - \frac{P_j^T 2 \tilde{X}}{P_j^T 3 \tilde{X}} \right)^2 \right) \quad (4)$$

where V_{ij} is the visibility matrix, C_i and q_i are camera positions and orientations (represented as quaternions), and X_j are 3D point positions. To address this large-scale optimization problem efficiently, we use the `scipy.optimize.least_squares` function along with Sparse Bundle Adjustment toolboxes such as large-scale BA in `scipy` which are designed to find a solution for such optimization by exploiting the sparsity of the visibility matrix.

The output after Bundle Adjustment is shown in Figure 8

L. Results and Conclusion

Our implementation of the Structure from Motion pipeline effectively reconstructed 3D scenes from 2D image sequences.

The iterative refinement approach, which culminated in bundle adjustment increased the accuracy of our reconstruction. This optimization technique improved both camera poses and 3D point placements, dramatically reducing reprojection errors across all perspectives. The findings highlight the significance of bundle adjustment in producing high-quality 3D reconstructions, as it coherently combines information from all images and points, minimizing errors from earlier stages of the pipeline.

Method	Mean Error
Linear Triangulation	1.4918
Non-linear Triangulation	1.4190
PnP	11.9391
Non-linear PnP	10.0525

TABLE I
MEAN ERROR COMPARISON FOR DIFFERENT SfM STEPS

II. EXTRA CREDIT

We used an enhanced Structure from Motion (SfM) pipeline to process our own dataset, proving the flexibility and resilience of our technique. To achieve the best results, we concentrated our data gathering strategy on capturing high-quality images of a building under controlled conditions. We used a smartphone camera in manual mode, with constant settings for aperture(F1.7), shutter speed(1/1000 s), WB(5000K) and ISO(50) throughout all photographs. The data was taken under neutral lighting circumstances, in the evening, to reduce strong shadows and provide uniform illumination throughout the subject.

To prepare our dataset, we calibrated the camera first. For this phase we used the metadata from the image and calculated approximate intrinsic camera characteristics. For feature recognition and matching, we used the Scale-Invariant Feature Transform (SIFT) technique with high sensitivity settings. This decision was based on SIFT’s strong performance, faster feature detection techniques. To ensure compatibility with our existing SfM pipeline, we saved the resulting feature matches in the same format as our original dataset, naming the files `matching1.txt` through `matching4.txt`. The images were originally 4000x3000 pixel resolution but were scaled down to 800x600 pixels similar to dataset in the assignment.

Our SfM implementation successfully processed this bespoke dataset, proving its capacity to deal with real-world, self-collected data. The pipeline contained all of the necessary steps: feature matching, fundamental matrix estimation, essential matrix calculation, camera position estimation, triangulation, Perspective-n-Point and bundle adjustment.

The results for our bespoke dataset were impressive, confirming the strength and precision of our Structure from Motion (SfM) methodology. By plotting the rebuilt 3D coordinates, we were able to see the building’s exact geometry. Notably, the triangular arch at the top of the building was flawlessly replicated, demonstrating the accuracy of our reconstruction approach. This geometrically complicated feature

demonstrated our pipeline’s ability to capture minute structural details. The successful reconstruction validates our implementation’s stability and capacity to handle real-world datasets with complicated architectural aspects.

The results of our SfM pipeline applied to our custom dataset are shown in the following figures:

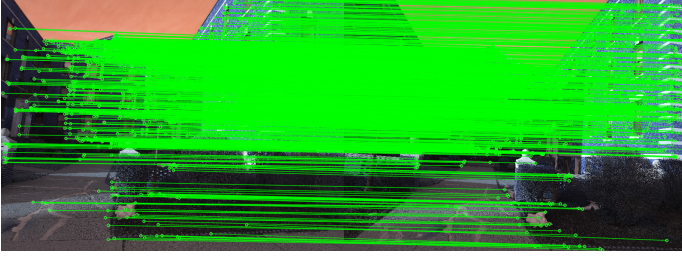


Fig. 9. RANSAC Matches

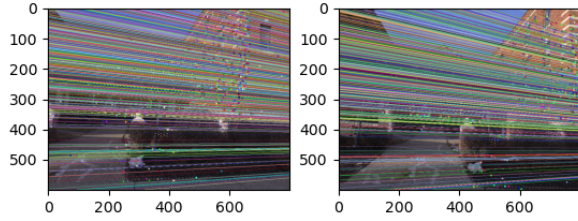


Fig. 10. Epipolar Lines

III. PHASE 2 : DEEP LEARNING APPROACH NERF

A. Introduction

Neural Radiance Fields (NeRF) uses deep learning techniques to create highly detailed and precise three-dimensional models from a collection of two-dimensional photographs. This method entails training a neural network to predict the brightness and density of points in 3D space, so efficiently capturing the geometry and look of a scene. NeRF creates fresh perspectives from any perspective by projecting rays through each pixel in the input images, resulting in remarkable realism and immersion. NeRF’s applications span multiple disciplines, allowing it to streamline the generation of photorealistic settings and objects, revolutionizing digital media production.

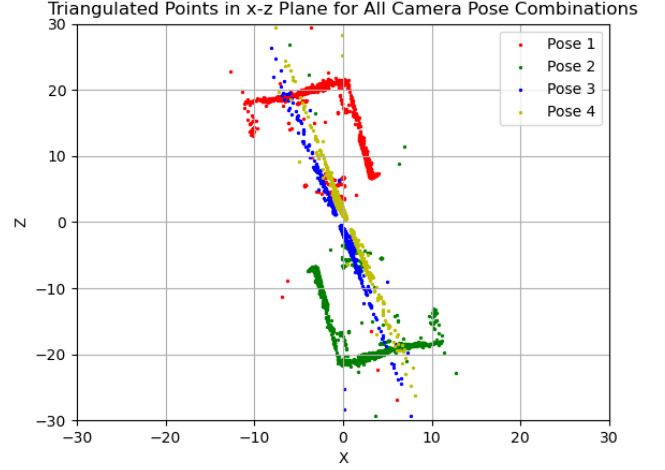


Fig. 11. Linear Triangulation

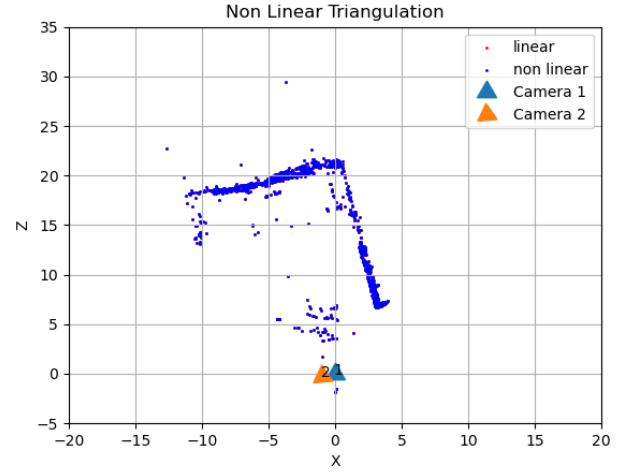


Fig. 12. Non Linear Triangulation

B. NeRF Pipeline

NeRF represents a scene as a continuous 5D function, mapping each spatial position (x, y, z) and viewing direction (θ, ϕ) to an emitted color (r, g, b) and volume density σ .

$$f_{\Theta} : (x, y, z, \theta, \phi) \rightarrow (r, g, b, \sigma) \quad (5)$$

The function is built using a fully-connected neural network (MLP) with parameters Θ . The MLP architecture is made up of eight fully connected layers with ReLU activations, a ninth layer of dimension 256, and a final layer with 128 nodes and three output dimensions for RGB values. A different branch after the seventh layer outputs the volume density σ .

To allow the network to express high-frequency changes, NeRF uses positional encoding on the input coordinates:

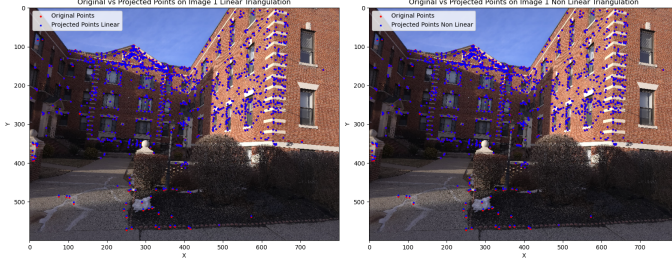


Fig. 13. Projected points after Linear vs Non Linear Triangulation

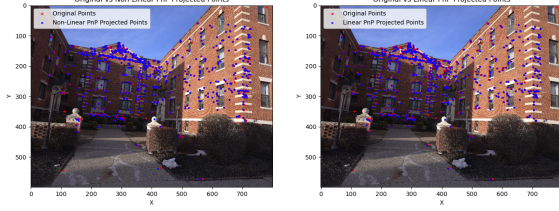


Fig. 14. Projected Points after after Linear vs Non Linear PnP

$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p)) \quad (6)$$

This encoding converts the input coordinates to a higher-dimensional space, allowing the network to detect tiny details in the picture.

NeRF creates new vistas utilizing standard volume rendering techniques. For a camera ray $r(t) = \mathbf{o} + t\mathbf{d}$ with origin $\mathbf{o}$ and direction $\mathbf{d}$, the predicted color $C(r)$ is determined as:

$$C(r) = \int_{t_n}^{t_f} T(t) \sigma(r(t)) c(r(t), \mathbf{d}) dt \quad (7)$$

where $T(t)$ is the accumulated transmittance:

$$T(t) = \exp \left(- \int_{t_n}^t \sigma(r(s)) ds \right) \quad (8)$$

In practice, this integral is approximated by taking discrete samples along each ray.

$$C(r) = \sum_{n=1}^N T_n \cdot (1 - \exp(-\sigma_n \delta_n)) \cdot c_n \quad (9)$$

where:

$$T_n = \exp \left(- \sum_{k=1}^{n-1} \sigma_k \delta_k \right) \quad (10)$$

Here, δ_n is the distance between adjacent samples, σ_n is the volume density at sample point n , and c_n is the color at that point.

The training procedure requires only a series of photographs with established camera poses. For each training image, rays are drawn through the pixels, and points are sampled along them. The network predicts the colors and densities of these

Triangulated Points in x-z Plane for All Camera Pose Combinations using PnP

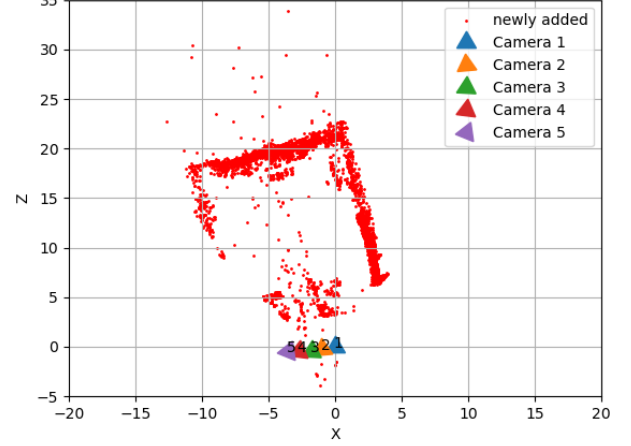


Fig. 15. Camera Poses and Projected Points After Non Linear PnP

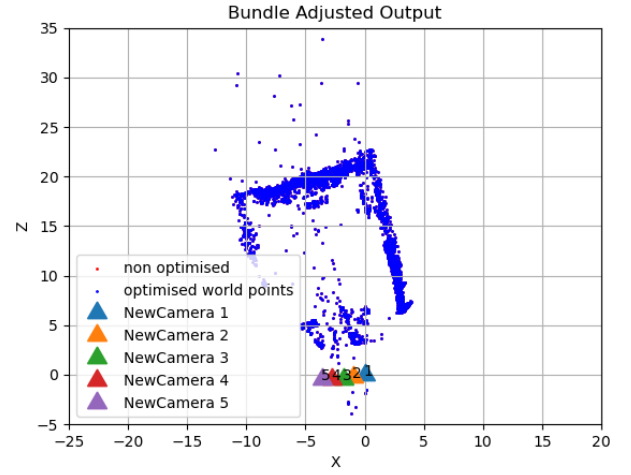


Fig. 16. Final Bundle Adjustment Output

dots, which are then converted into an image using the volume rendering equation.

The network is optimized by reducing the L2 loss between the displayed and ground truth images:

$$\mathcal{L} = \sum_{\mathbf{r} \in \mathcal{R}} |\hat{C}(\mathbf{r}) - C_{gt}(\mathbf{r})|_2^2 \quad (11)$$

where $\mathcal{R}$ is the set of rays in the batch; $\hat{C}(\mathbf{r})$ is the predicted color for ray $\mathbf{r}$; and $C_{gt}(\mathbf{r})$ is the ground truth color.

C. Network Architecture

The neural network architecture illustrated here uses two different Neural Radiance Fields (NeRF) implementations. The primary model uses positional encoding to improve the depiction of high-frequency information. This design comprises of an 8-layer MLP with ReLU activations and a skip connection at the fifth layer, which concatenates the encoded

position input with intermediate features. The network processes spatial coordinates (x, y, z) through positional encoding $\gamma(x)$, which maps each coordinate to a higher-dimensional space using sinusoidal functions at multiple frequency bands. This transforms 3D inputs into $(6L + 3)$ -dimensional vectors, where L represents the number of frequency bands. The encoded locations pass through the network to yield volume density σ and a feature vector. These are then merged with similarly encoded viewing directions (with default $L = 4$) to generate RGB color outputs via subsequent layers. The network architecture is displayed in Figure 17.

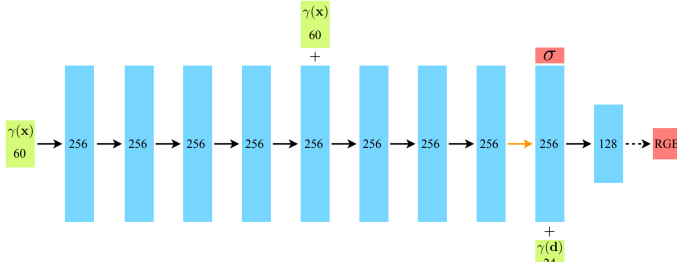


Fig. 17. Network Architecture

The second form (NeRFmodel2) uses the same architectural skeleton but functions directly on raw coordinates, without positional encoding. This simplified model retains the 8-layer MLP structure, including ReLU activations and a skip connection at layer 5, but processes raw 3D position information directly. Similarly, it processes raw viewing directions without frequency encoding and concatenates them with the feature vector before making the final RGB forecast. Both models use sigmoid activation for the RGB output to limit values to 0 to 1, and ReLU activation for the density output to assure non-negative values. This architecture comparison permits an examination of the effect of positional encoding on the network’s capacity to represent high-frequency details in 3D scenes.

D. Training

In our implementation of the NeRF model, we trained the network using a well developed optimization procedure. We started our model with or without positional encoding (depending on the configuration parameter) and trained it with the Adam optimizer, an initial learning rate of $5e-4$, and a multi-step learning rate scheduler. Initially, we attempted to employ $32 \times 32 \times 4$ (4096) rays each batch, however this configuration resulted in exorbitant training times—approximately 6 hours for just 25,000 iterations, or around 3 days for complete training. Given our project’s timetable and available resources, this computing cost was prohibitively high. We reduced batch size to 32×32 (1024) rays, allowing us to finish training in around 22 hours. This trade-off between computational efficiency and model performance was required to achieve project deadlines while producing acceptable results. During training, we randomly selected rays from the training images, and for each ray, we sampled 400 points stratified by near



Fig. 18. Lego - Training Loss (with positional encoding)

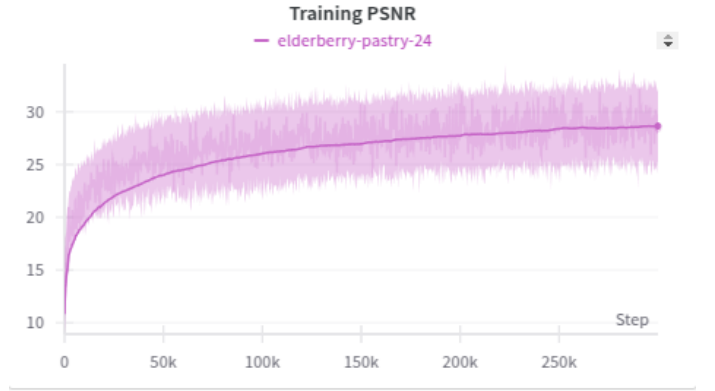


Fig. 19. Lego - Training PSNR (with positional encoding)

and distant planes (2.0 and 6.0 units), determining the volume density and color at each point. The model was trained to minimize the mean squared error between rendered and ground truth pixel colors, with checkpoints maintained every 1000 iterations, and the best model was kept based on the lowest loss. Throughout training, we tracked both loss and PSNR metrics using Weights and Biases for visualization. The plots of Training loss and Training PSNR for Lego and Ship dataset are displayed in Figures 18, 19, 20, 21, 22, 23, 24, 25.

E. Testing

To test the NeRF model, we generated unique views by throwing rays through each pixel of the test images. To manage GPU memory limits, we processed the rays in 4096 chunks and rendered the image progressively. For each test pose, we produced rays for all pixels in the image, computed the color along each ray using our trained model, and then constructed the final rendered image. We assessed the quality of our renderings using two metrics: peak signal-to-noise ratio (PSNR) and structural similarity index measure (SSIM), comparing the produced images to the ground truth. The testing process ended with saving the produced images and calculating the



Fig. 20. Lego - Training Loss (without positional encoding)



Fig. 23. Ship - Training PSNR (with positional encoding)



Fig. 21. Lego - Training PSNR (without positional encoding)

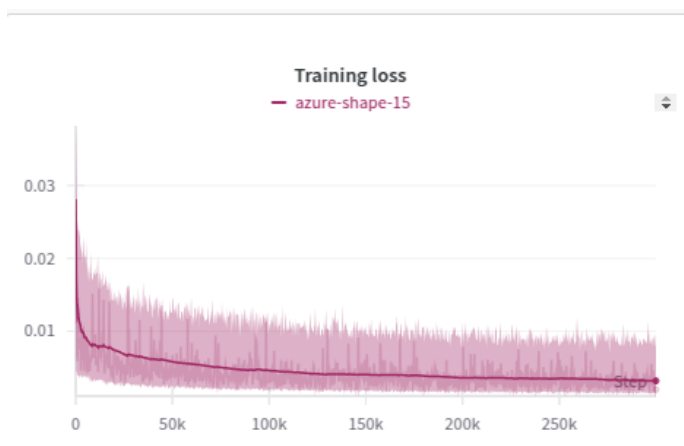


Fig. 24. Ship - Training Loss (without positional encoding)

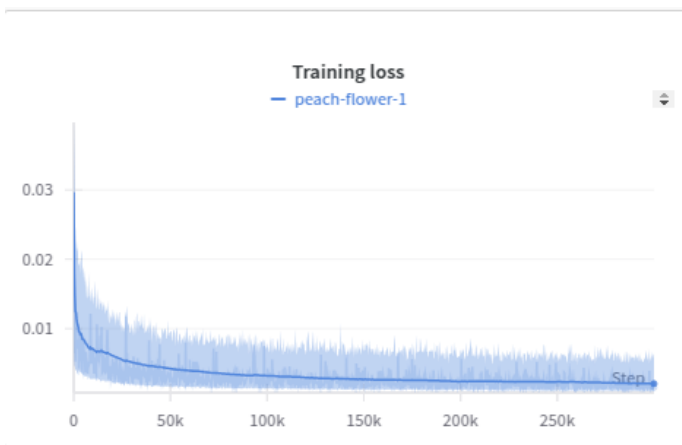


Fig. 22. Ship - Training Loss (with positional encoding)



Fig. 25. Ship - Training PSNR (without positional encoding)

average PSNR and SSIM for all test views. In addition, we built an animated GIF from the sequence of rendered photos to demonstrate the model’s potential to provide unique views of the 3D scene. The NeRF based reconstruction with and without positional encoding and comparison with Ground Truth is displayed in Figure 26.

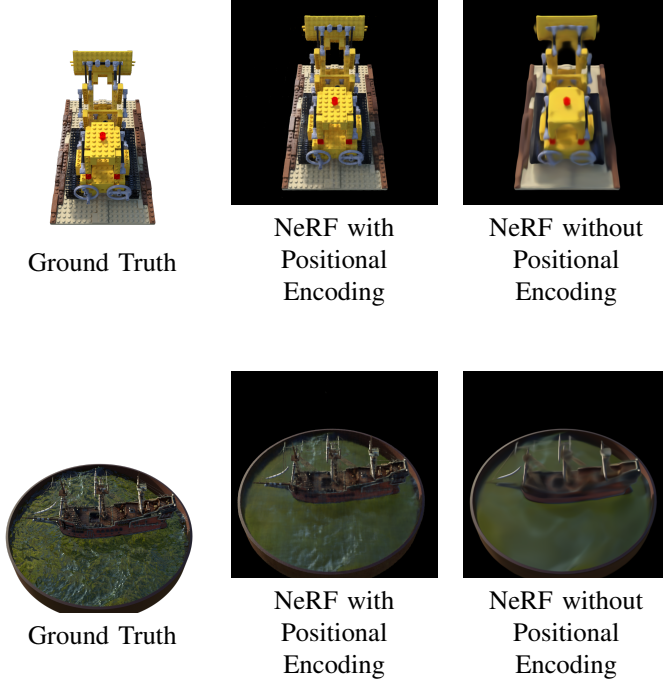


Fig. 26. Ground Truth vs NeRF Output Comparison

F. Results and Conclusion

Our implementation of the Neural Radiance Fields (NeRF) model performed admirably on both the Lego and ship datasets, especially when using positional encoding. The model with positional encoding consistently provided better visual quality, with higher PSNR and SSIM metrics than the variation without positional encoding. This performance difference is due to the inherent restriction of typical MLPs in modeling high-frequency functions. Positional encoding allows the network to capture fine-grained details and complex surface textures that would otherwise be smoothed away by mapping input coordinates to a higher-dimensional space using sinusoidal functions at several frequency bands. The transformation of 3D inputs into $(6L + 3)$ -dimensional vectors, where L represents the number of frequency bands (10 for positions and 4 for viewing directions in our implementation), gives the network better representational capacity for high-frequency variations in geometry and appearance, which is especially important for accurately reconstructing complex objects with intricate details like Lego and ship models.

IV. EXTRA CREDIT NeRF

In order to assess the robustness of Neural Radiance Fields beyond common benchmark datasets, we created and processed a new dataset using an astronaut figurine. The data

TABLE II
AVERAGE TEST METRICS

Model	Lego (PosEnc)	Lego (w/o PosEnc)	Ship (PosEnc)	Ship (w/o PosEnc)
SSIM	0.91	0.83	0.82	0.79
PSNR	28.4	23.91	26.43	24.50

collection procedure begins with taking several photographs of the topic with Polycam, a photogrammetry tool designed to allow for 3D reconstruction from 2D images. This method provided us with a broad variety of perspectives required for comprehensive scene representation, while also allowing us to determine if NeRF could effectively reconstruct real-world objects taken in non-laboratory settings.

Following picture acquisition, we analyzed our dataset using COLMAP, an open-source Structure-from-Motion (SfM) and Multi-View Stereo (MVS) pipeline. COLMAP used feature extraction, matching, and bundle adjustment to estimate camera postures and intrinsic parameters in each image. We then created a custom Python script to convert COLMAP’s output into a JSON format that is suitable with our NeRF implementation, translating the sparse reconstruction data into the camera transformation matrices and intrinsic parameters needed by our rendering process. This conversion was a vital step toward bridging the gap between general-purpose photogrammetry tools and our specialized neural rendering framework.

Our early training attempts on the unprocessed dataset produced unsatisfactory results, including fuzzy reconstructions and artifacts. Analysis found that the inclusion of complex, inconsistent backgrounds in our photographs greatly reduced the model’s capacity to learn accurate scene representations. As a result, we repeated the same process with one important modification: we removed the background from all photographs after COLMAP processing. This preprocessing phase was included to help the model focus solely on the subject of interest, removing any potential distraction from unrelated ambient factors.

The background-removed dataset provided significantly better reconstructions than our first approach, with more defined geometry and consistent appearance across novel viewpoints. However, the results fell short of the quality shown in benchmark datasets such as Lego and Ship. The average PSNR value was found to be 31.49 and the average SSIM value was 0.94 for this custom dataset. We attribute this performance gap to a number of factors, including potential inconsistencies introduced during the background removal process, which may have resulted in artificial edges or lighting discontinuities; variations in lighting conditions during photography that violated NeRF’s assumption of static illumination; and possibly insufficient image coverage of specific regions of the astronaut figurine. These findings show the difficulties in applying NeRF to real-world, user-captured data and emphasize the significance of regulated data collecting techniques for peak performance. Our result for the custom dataset is shown in Figure 28.



Fig. 27. Custom Dataset Ground Truth



Fig. 28. Custom Dataset NeRF output

V. CHECKPOINT AND EXTRA CREDIT

For Phase2 of the project, we have organized additional resources into Nerf_phase2 folder: the **Checkpoints folder** and the **Data_custom folder**. These folders are essential for reproducing the results and running the code as described in the README file.

Available at: [here](#)

A. Checkpoints Folder

- **Location:** Place the entire Checkpoints folder inside the Phase2 directory.
- **Contents:** The checkpoints folder contains five checkpoint folder which have final model for each of them:
 - example_checkpoint_custom
 - example_checkpoint_lego
 - example_checkpoint_lego_no_pos
 - example_checkpoint_ship
 - example_checkpoint_ship_no_pos
- **Usage:** To load the desired checkpoint during model initialization, pass the appropriate argument when running the code. Refer to the README file for specific argument options. For example, use an argument like: `--checkpoint_path`

`./Checkpoints/example_checkpoint_ship/` to load the Ship model with positional encoding.

B. Data_custom Folder

- **Location:** Add the Data_custom folder to the Phase2 directory alongside the checkpoint folder.
- **Contents:** This folder contains a data subfolder with additional datasets for extra credit experiments.
- **Usage:** Follow the instructions in the README file to run the code associated with this folder.

REFERENCES

- [1] Agarwal, S., Snively, N., Simon, I., Seitz, S. M., & Szeliski, R. (2009). *Building Rome in a Day*. Proceedings of the IEEE International Conference on Computer Vision (ICCV). Available at: https://grail.cs.washington.edu/rome/rome_paper.pdf
- [2] Hartley, R., & Zisserman, A. (2003). *Multiple View Geometry in Computer Vision*. Cambridge University Press, Second Edition. Available at: https://www.r-5.org/files/books/computers/algo-list/image-processing/vision/Richard_Hartley_Andrew_Zisserman-Multiple_View_Geometry_in_Computer_Vision-EN.pdf
- [3] SciPy Cookbook: Bundle Adjustment Example. Available at: https://scipy-cookbook.readthedocs.io/items/bundle_adjustment.html
- [4] Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2020). *Representing Scenes as Neural Radiance Fields for View Synthesis*. arXiv preprint arXiv:2003.08934. Available at: <https://arxiv.org/pdf/2003.08934.pdf>

- [5] Polycam. (2025). *Capture reality with Polycam's LiDAR scanner & photogrammetry platform*. Available at: <https://poly.cam/>
- [6] COLMAP. (n.d.). *COLMAP – A General-Purpose Structure-from-Motion (SfM) and Multi-View Stereo (MVS) Pipeline with a Graphical and Command-Line Interface*. Available at: <https://colmap.github.io/>
- [7] Gatis, D. (2020). *rembg: A tool to remove image backgrounds using Python*. GitHub repository. Available at: <https://github.com/danielgatis/rembg>